Lab 7: Fuzzing for Input Validation Bugs

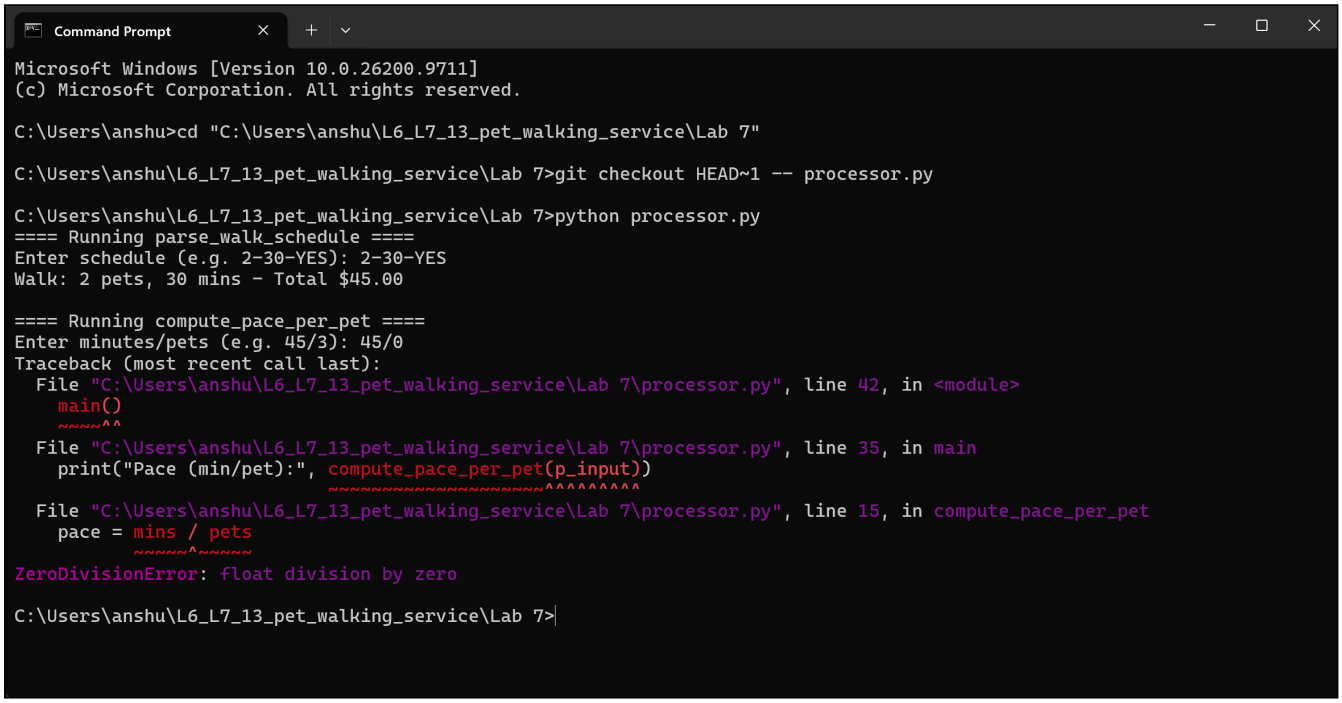
Application: L6_L7_13 Pet Walking Service | **Tool:** Hypothesis 6.140 + pytest 9.2 on Python 3.13.6
Name: Anshul Poovaiah K | **SRN:** PES1UG24CS071

Step 1: Dependencies

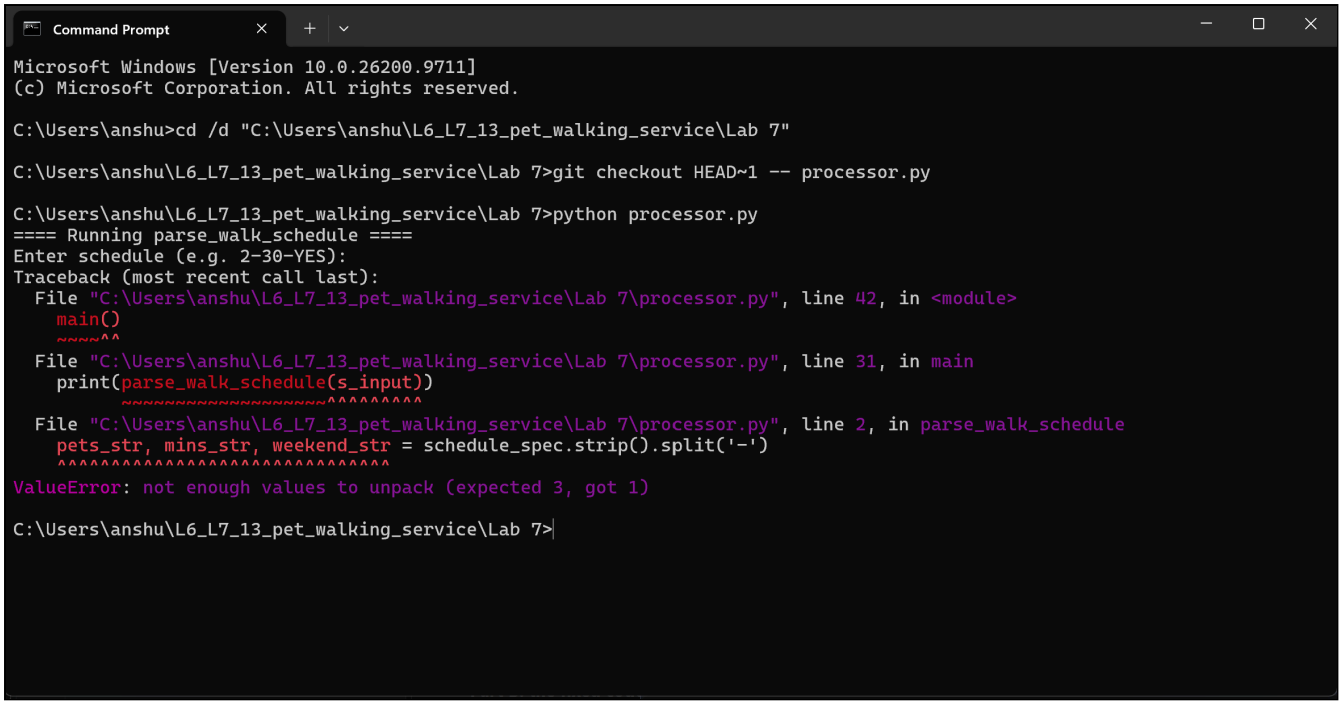
```
$ pip install hypothesis
$ pip install pytest
```

Step 2: Execution of the original (buggy) application

Run 1: the schedule `2-30-YES` works, but entering `45/0` for the pace crashes the program with `ZeroDivisionError`.

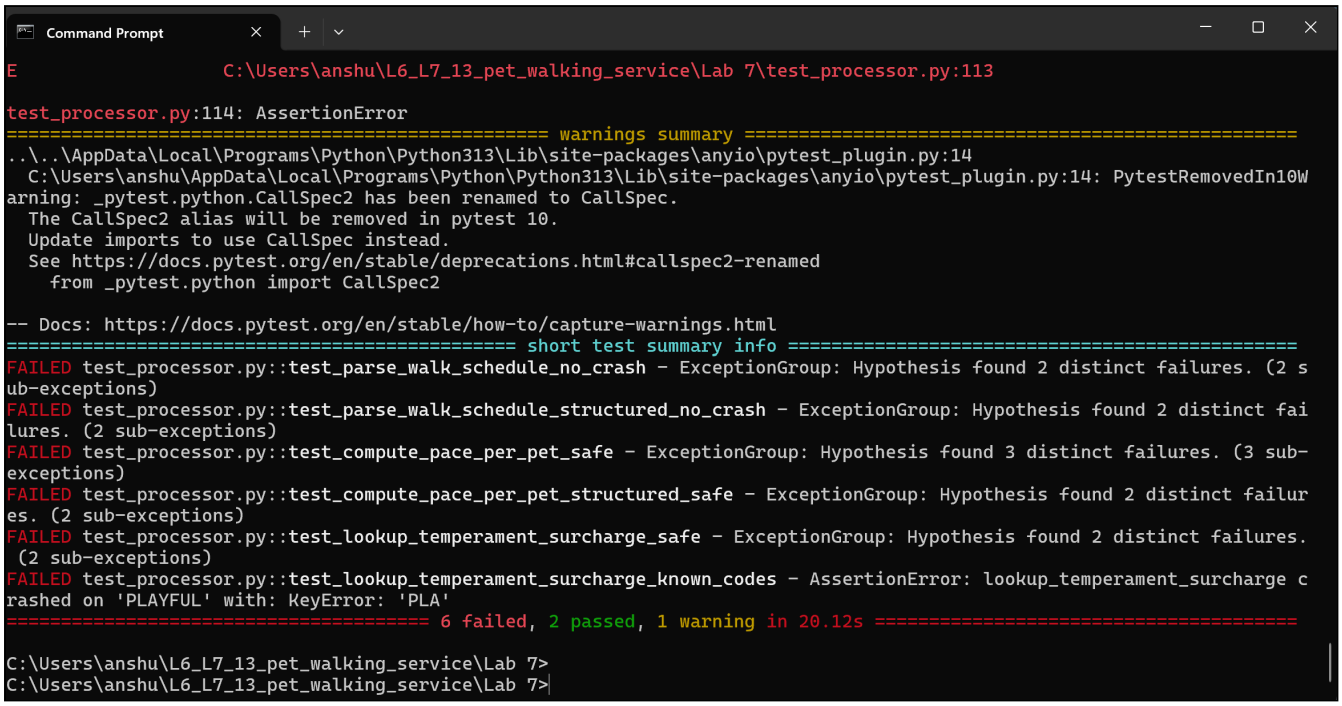


Run 2: pressing Enter without typing a schedule also crashes the original program (`ValueError`).


Command Prompt
Microsoft Windows [Version 10.0.26200.9711]
(c) Microsoft Corporation. All rights reserved.
C:\Users\anshu>cd /d "C:\Users\anshu\L6_L7_13_pet_walking_service\Lab 7"
C:\Users\anshu\L6_L7_13_pet_walking_service\Lab 7>git checkout HEAD~1 -- processor.py
C:\Users\anshu\L6_L7_13_pet_walking_service\Lab 7>python processor.py
==== Running parse_walk_schedule ====
Enter schedule (e.g. 2-30-YES):
Traceback (most recent call last):
File "C:\Users\anshu\L6_L7_13_pet_walking_service\Lab 7\processor.py", line 42, in <module>
main()
~~~~^^
File "C:\Users\anshu\L6_L7_13_pet_walking_service\Lab 7\processor.py", line 31, in main
print(parse_walk_schedule(s_input))
~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^
File "C:\Users\anshu\L6_L7_13_pet_walking_service\Lab 7\processor.py", line 2, in parse_walk_schedule
pets_str, mins_str, weekend_str = schedule_spec.strip().split('-')
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: not enough values to unpack (expected 3, got 1)
C:\Users\anshu\L6_L7_13_pet_walking_service\Lab 7>

Step 3/4: Fuzz tests run against the original code (failing)

Hypothesis generated random and malformed input (`st.text() | st.none()`) plus structured near-valid input. **6 of 8 tests failed**. The bug table below lists the inputs that broke each test.

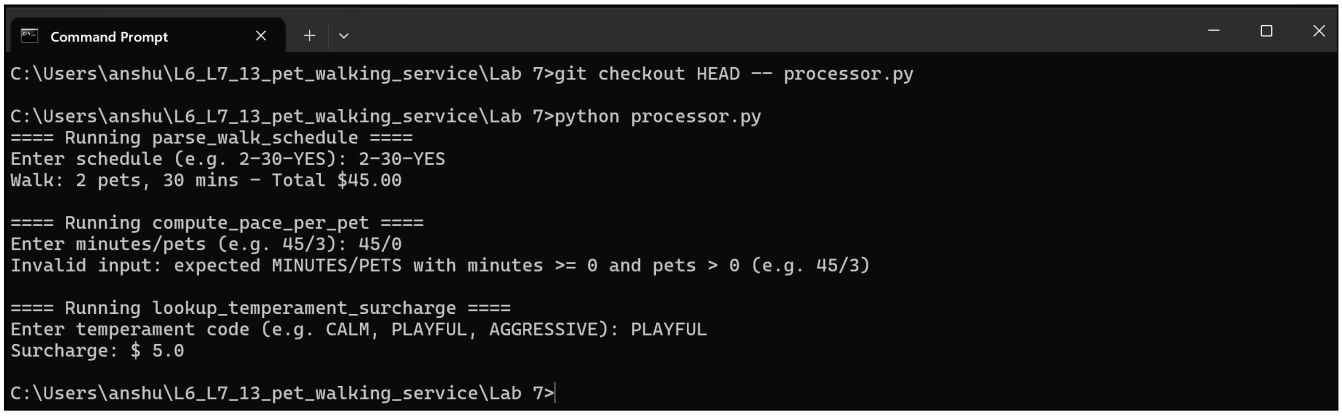


Bugs found by fuzzing and how they were fixed

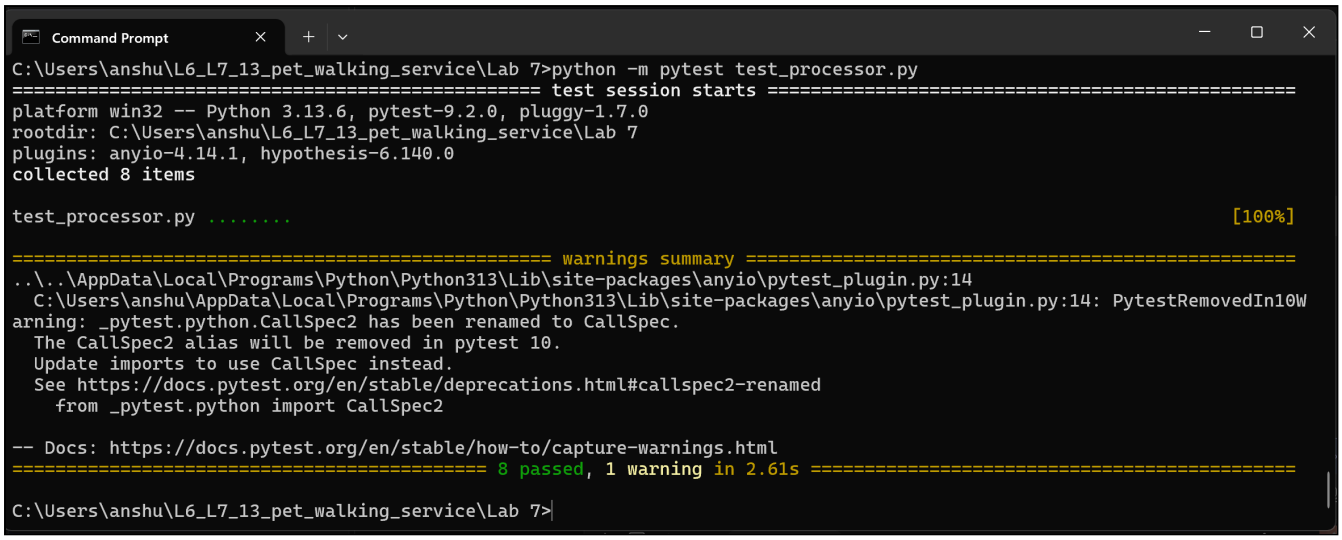
Function	Fuzzed input	Original behaviour	Fix
`parse_walk_schedule`	None input	AttributeError: 'NoneType' has no attribute 'strip'	Type check, returns an error message
`parse_walk_schedule`	'' / wrong number of '-' (e.g. '0--0--YES')	ValueError: not enough / too many values to unpack	Check exactly 3 parts
`parse_walk_schedule`	Non-numeric, zero, negative or huge pets/minutes	ValueError from int(), or nonsense prices	Digits only, range 1-10 pets, 1-240 mins
`parse_walk_schedule`	Weekend flag other than YES/NO	KeyError	dict.get() + error message
`compute_pace_per_pet`	None input	AttributeError: 'NoneType' has no attribute 'split'	Type check, returns None
`compute_pace_per_pet`	'', '/', '0/' (missing parts)	ValueError (unpack / float conversion)	Check 2 parts, try/except ValueError
`compute_pace_per_pet`	'0/0', '45/0'	ZeroDivisionError	Require pets > 0
`compute_pace_per_pet`	'1/-1', 'nan/1', 'inf/1'	Negative / nan / inf pace returned silently	Require finite values, mins >= 0
`lookup_temperament_surcharge`	None input	AttributeError: 'NoneType' has no attribute 'strip'	Type check, returns None
`lookup_temperament_surcharge`	'' or unknown code	KeyError	dict.get() returns None
`lookup_temperament_surcharge`	'PLAYFUL' (the prompt's own example)	KeyError: 'PLA' (table only had 'PLY')	Added 'PLA' mapping

Step 5: Execution of the corrected application

The same inputs as before (`2-30-YES`, `45/0`, `PLAYFUL`). Invalid input now gets a clear message and nothing crashes.



Step 6: Fuzz tests run against the fixed code (all passing)



All 8 tests pass. The 1 warning comes from a third-party pytest plugin (anyio) and is unrelated to the tests.

Source code: processor.py (fixed)

```
import math

# Business limits used for input validation
MAX_PETS = 10          # max pets in a single walk
MAX_MINS = 240         # max walk length in minutes
MAX_INPUT_LEN = 100    # reject absurdly long inputs (DoS / int-parsing protection)


def _parse_positive_int(text, max_value):
    """Return text as an int in [1, max_value], or None if invalid."""
    text = text.strip()
    # Only plain ASCII digits: rejects '', '+3', '-2', '3.5', '1_000', unicode digits, etc.
    if not text or not text.isascii() or not text.isdigit():
        return None
    value = int(text)
    if value < 1 or value > max_value:
        return None
    return value


def parse_walk_schedule(schedule_spec):
    """Parse 'PETS-MINUTES-YES/NO' and return a price summary, or an error message."""
    if not isinstance(schedule_spec, str):
        return "Invalid schedule: input must be a string"
    if len(schedule_spec) > MAX_INPUT_LEN:
        return "Invalid schedule: input too long"

    parts = schedule_spec.strip().split('-')
    if len(parts) != 3:
        return "Invalid schedule: expected format PETS-MINUTES-YES/NO (e.g. 2-30-YES)"
    pets_str, mins_str, weekend_str = parts

    pets = _parse_positive_int(pets_str, MAX_PETS)
    if pets is None:
        return f"Invalid schedule: pets must be a whole number from 1 to {MAX_PETS}"

    mins = _parse_positive_int(mins_str, MAX_MINS)
    if mins is None:
        return f"Invalid schedule: minutes must be a whole number from 1 to {MAX_MINS}"

    weekend_map = {"YES": 15.0, "NO": 0.0}
    weekend_fee = weekend_map.get(weekend_str.strip().upper())
    if weekend_fee is None:
        return "Invalid schedule: weekend flag must be YES or NO"

    base_fee = 20.0 if mins <= 30 else 35.0
    total = base_fee + (pets * 5.0) + weekend_fee
    return f"Walk: {pets} pets, {mins} mins - Total ${total:.2f}"


def compute_pace_per_pet(walk_metrics_str):
    """Parse 'MINUTES/PETS' and return minutes per pet, or None if invalid."""
    if not isinstance(walk_metrics_str, str) or len(walk_metrics_str) > MAX_INPUT_LEN:
        return None

    parts = walk_metrics_str.split('/')
    if len(parts) != 2:
        return None

    try:
        mins = float(parts[0])
        pets = float(parts[1])
    except ValueError:
        return None
```

```
    # Reject nan / inf, negative minutes, and zero or negative pets (division by zero)
    if not (math.isfinite(mins) and math.isfinite(pets)):
        return None
    if mins < 0 or pets <= 0:
        return None

    pace = mins / pets
    if not math.isfinite(pace):   # e.g. huge value divided by a tiny one overflows
        return None
    return round(pace, 2)


def lookup_temperament_surcharge(temperament_code):
    """Return the surcharge for a temperament code, or None if unknown."""
    if not isinstance(temperament_code, str):
        return None

    prefix = temperament_code.strip().upper()[:3]
    surcharges = {
        "CAL": 0.0,   # CALM
        "PLY": 5.0,   # PLY (short code)
        "PLA": 5.0,   # PLAYFUL  (original code crashed: 'PLAYFUL'[:3] == 'PLA')
        "REA": 12.0,  # REACTIVE
        "AGG": 20.0   # AGGRESSIVE
    }
    return surcharges.get(prefix)


def main():
    print("==== Running parse_walk_schedule ====")
    s_input = input("Enter schedule (e.g. 2-30-YES): ")
    print(parse_walk_schedule(s_input))

    print("\n==== Running compute_pace_per_pet ====")
    p_input = input("Enter minutes/pets (e.g. 45/3): ")
    pace = compute_pace_per_pet(p_input)
    if pace is None:
        print("Invalid input: expected MINUTES/PETS with minutes >= 0 and pets > 0 (e.g. 45/3)")
    else:
        print("Pace (min/pet):", pace)

    print("\n==== Running lookup_temperament_surcharge ====")
    t_input = input("Enter temperament code (e.g. CALM, PLAYFUL, AGGRESSIVE): ")
    surcharge = lookup_temperament_surcharge(t_input)
    if surcharge is None:
        print("Invalid input: unknown temperament code (use CALM, PLAYFUL, REACTIVE or AGGRESSIVE)")
    else:
        print("Surcharge: $", surcharge)


if __name__ == "__main__":
    main()
```

Source code: test_processor.py

```
import math

from hypothesis import given, settings, strategies as st
from processor import parse_walk_schedule, compute_pace_per_pet, lookup_temperament_surcharge


# ---------------------------------------------------------------------------
# Strategies
# ---------------------------------------------------------------------------

# Completely random / malformed input (the core fuzzing strategy).
any_input = st.text() | st.none()

# "Almost valid" inputs that look like real data, so the fuzzer also reaches
# the logic after the parsing step (zero/negative numbers, bad flags, etc.).
schedule_like = st.builds(
    lambda p, m, w, sep: f"{p}{sep}{m}{sep}{w}",
    st.integers(min_value=-100, max_value=100),
    st.integers(min_value=-100, max_value=500),
    st.sampled_from(["YES", "NO", "yes", " no ", "MAYBE", ""]),
    st.sampled_from(["-", "--", " - "]),
)

pace_like = st.builds(
    lambda m, p: f"{m}/{p}",
    st.floats(allow_nan=True, allow_infinity=True) | st.integers(-100, 100),
    st.floats(allow_nan=True, allow_infinity=True) | st.integers(-100, 100),
)

temperament_like = st.sampled_from(
    ["CALM", "PLAYFUL", "REACTIVE", "AGGRESSIVE", "calm", "  ply ", "", "XYZ", "AG"]
)


# ---------------------------------------------------------------------------
# 1. parse_walk_schedule
# ---------------------------------------------------------------------------

@given(any_input)
def test_parse_walk_schedule_no_crash(s):
    try:
        result = parse_walk_schedule(s)
    except Exception as e:
        assert False, f"parse_walk_schedule crashed on {s!r} with: {type(e).__name__}: {e}"
    assert isinstance(result, str)


@given(schedule_like)
def test_parse_walk_schedule_structured_no_crash(s):
    try:
        result = parse_walk_schedule(s)
    except Exception as e:
        assert False, f"parse_walk_schedule crashed on {s!r} with: {type(e).__name__}: {e}"
    assert isinstance(result, str)
    # A schedule with zero or negative pets/minutes must never be priced.
    if "Total" in result:
        assert "-" not in result.split("Total")[1], f"Negative total for {s!r}: {result}"


@given(st.integers(1, 10), st.integers(1, 240), st.sampled_from(["YES", "NO"]))
def test_parse_walk_schedule_valid_total(pets, mins, weekend):
    result = parse_walk_schedule(f"{pets}-{mins}-{weekend}")
    expected = (20.0 if mins <= 30 else 35.0) + pets * 5.0 + (15.0 if weekend == "YES" else 0.0)
    assert result == f"Walk: {pets} pets, {mins} mins - Total ${expected:.2f}"
```

```
# ---------------------------------------------------------------------------
# 2. compute_pace_per_pet
# ---------------------------------------------------------------------------

@given(any_input)
def test_compute_pace_per_pet_safe(s):
    try:
        result = compute_pace_per_pet(s)
    except Exception as e:
        assert False, f"compute_pace_per_pet crashed on {s!r} with: {type(e).__name__}: {e}"
    assert result is None or isinstance(result, float)


@given(pace_like)
def test_compute_pace_per_pet_structured_safe(s):
    try:
        result = compute_pace_per_pet(s)
    except Exception as e:
        assert False, f"compute_pace_per_pet crashed on {s!r} with: {type(e).__name__}: {e}"
    # Pace must be a real, non-negative number (no nan / inf / negative pace).
    if result is not None:
        assert math.isfinite(result) and result >= 0, f"Bad pace {result} for {s!r}"


@given(st.integers(0, 600), st.integers(1, 20))
def test_compute_pace_per_pet_valid(mins, pets):
    assert compute_pace_per_pet(f"{mins}/{pets}") == round(mins / pets, 2)


# ---------------------------------------------------------------------------
# 3. lookup_temperament_surcharge
# ---------------------------------------------------------------------------

@given(any_input)
def test_lookup_temperament_surcharge_safe(s):
    try:
        result = lookup_temperament_surcharge(s)
    except Exception as e:
        assert False, f"lookup_temperament_surcharge crashed on {s!r} with: {type(e).__name__}: {e}"
    assert result is None or result in (0.0, 5.0, 12.0, 20.0)


@given(temperament_like)
def test_lookup_temperament_surcharge_known_codes(s):
    try:
        result = lookup_temperament_surcharge(s)
    except Exception as e:
        assert False, f"lookup_temperament_surcharge crashed on {s!r} with: {type(e).__name__}: {e}"
    expected = {"CAL": 0.0, "PLY": 5.0, "PLA": 5.0, "REA": 12.0, "AGG": 20.0}
    assert result == expected.get(s.strip().upper()[:3])
```